
Algorithm 1 Merge ratio and score tables `build_id_data_dict`

Input: ratio table; score table; join type `merge_how`

Output: `id_data_dict[id] = (times, ratios, scores)`

- 1: `merged` $\leftarrow$ join the tables on (id, time) using `merge_how`
 - 2: **for** each (id, rows) in `merged` **do**
 - 3: sort rows by time
 - 4: `id_data_dict[id]` $\leftarrow$ (time, ratio, score)
 - 5: **end for**
 - 6: **return** `id_data_dict`
-

Algorithm 2 Encode a ratio `RatioFeatureEncoder`

Input: ratio x of shape $(\dots, 1)$

Output: embedding of size d_{model} ; a scalar score during pretraining

1: $h \leftarrow \text{ReLU}(\text{Linear}(1 \rightarrow \text{hidden_dim})(x))$

2: $\text{embedding} \leftarrow \text{Linear}(\text{hidden_dim} \rightarrow d_{\text{model}})(h)$

3: $\text{score} \leftarrow \text{Linear}(d_{\text{model}} \rightarrow 1)(\text{embedding})$ // pretraining only

4: **return** embedding from `get_feature`; (embedding, score) from the pretraining forward pass

Algorithm 3 Pretrain the encoder `pretrain_ratio_encoder`

Input: sequences of (time, ratio, score); epochs, lr, batch_size

Output: encoder in eval mode

```
1: pairs  $\leftarrow$  every (ratio, score) from every sequence // time is not an input
2: for epoch  $\leftarrow$  1 to epochs do
3:   for each minibatch of pairs do
4:     ( $\cdot$ , predicted_score)  $\leftarrow$  encoder(ratio)
5:     step Adam on mean_squared_error(predicted_score, score)
6:   end for
7: end for
8: encoder.eval()
9: return encoder
```

Algorithm 4 Build the Gaussian-process dataset `build_global_dataset`

Input: sequences; frozen encoder

Output: dataset of (x, score) , with x shaped $(1 + d_{\text{model}})$

- 1: **for** each (time, ratio, score) in every sequence **do**
 - 2: embedding $\leftarrow$ encoder.get_feature(ratio)
 - 3: $x \leftarrow$ concatenate(ratio, embedding)
 - 4: store (x, score)
 - 5: **end for**
 - 6: **return** the stacked dataset
-

Algorithm 5 Fit the deep-kernel Gaussian process `train_ocrp_model`

Input: `id_data_dict`; `widths`; learning rate; epoch counts

Output: checkpoint `OCRP_model.pth`

- 1: pretrain the encoder, then freeze every encoder parameter
 - 2: `dataset` $\leftarrow$ `build_global_dataset(encoder)`
 - 3: `(train_set, val_set)` $\leftarrow$ random split of dataset by `val_ratio`
 - 4: `likelihood` $\leftarrow$ `GaussianLikelihood`
 - 5: `gp` $\leftarrow$ `DeepKernelGP(trainx, trainy)`
 - 6: `feature` $\leftarrow$ ReLU MLP from $1 + d_{\text{model}}$ to d_{feat}
 - 7: `mean` $\leftarrow$ 0; `kernel` $\leftarrow$ `scale` $\times$ RBF with one length-scale per latent dimension
 - 8: **for** `epoch` $\leftarrow$ 1 **to** `n_epochs` **do**
 - 9: `loss` $\leftarrow$ `reduce(-exact_marginal_log_likelihood(gp(trainx), trainy))`
 - 10: step Adam on the GP and the likelihood
 - 11: **end for**
 - 12: save encoder, GP, likelihood, `trainx`, `trainy`, and config
 - 13: **return** the checkpoint path
-

Algorithm 6 Predict the optimal ratio `predict_optimal_ratio`

Input: GP; frozen encoder; interval [low, high]; n_{starts} ; n_{steps}

Output: (best_ratio, best_mean)

```
1: best_mean  $\leftarrow -\infty$ 
2: for each start in linspace(low, high,  $n_{\text{starts}}$ ) do
3:   ratio  $\leftarrow$  start
4:   for step  $\leftarrow 1$  to  $n_{\text{steps}}$  do
5:      $x \leftarrow$  concatenate(ratio, encoder.get_feature(ratio))
6:     step Adam on  $-\text{gp}(x).\text{mean}$ 
7:     clamp ratio to [low, high]
8:   end for
9:   if  $\text{gp}(\text{ratio}).\text{mean} > \text{best\_mean}$  then
10:    keep this ratio
11:   end if
12: end for
13: return best_ratio, best_mean
```

Algorithm 7 Recommend the next ratio `ucb_acquisition`

Input: GP; likelihood; frozen encoder; interval [low, high]; $n_{\text{candidates}}$; β

Output: (best_ratio, best_ucb)

```
1: best_ucb  $\leftarrow -\infty$ 
2: for each ratio in linspace(low, high,  $n_{\text{candidates}}$ ) do
3:    $x \leftarrow \text{concatenate}(\text{ratio}, \text{encoder.get\_feature}(\text{ratio}))$ 
4:   posterior  $\leftarrow \text{gp}(x)$ 
5:   ucb  $\leftarrow \text{posterior.mean} + \beta \cdot \text{posterior.std}$ 
6:   if ucb > best_ucb then
7:     keep this ratio
8:   end if
9: end for
10: return best_ratio, best_ucb
```

Algorithm 8 Fit one cell group `RatioTimePredictor.fit_id`

Input: id; times; ratios; fit_ode; model_type

Output: stored residual model, kinetic parameters, and history

```
1: if the series has fewer than min_points then
2:   return skip this id
3: end if
4: if fit_ode then
5:   parameters  $\leftarrow$  differential evolution on the ODE mean squared error
6:   if that search fails then
7:     parameters  $\leftarrow$  bounded local minimization, or the defaults
8:   end if
9: else
10:  parameters  $\leftarrow$  default kinetics
11: end if
12: ode_ratio  $\leftarrow$  BxB1 ODE at each observed time
13: residual  $\leftarrow$  ratios - ode_ratio
14: if model_type is gpr then
15:   fit a Gaussian process from time to residual
16: else
17:   fit a polynomial from time to residual
18: end if
19: store the residual model, parameters, and (times, ratios)
```

Algorithm 9 Query time to the target ratio `RatioTimePredictor.query`

Input: `id`; `target_ratio`; `max_time`

Output: (`found`, `time`)

```
1: if some historical  $|\text{ratio} - \text{target\_ratio}| \leq \text{tolerance}$  then
2:   return the earliest such time
3: end if
4: if the target is above every observed ratio and the last slope  $\leq 0$  then
5:   return not found
6: end if
7: if the target is below every observed ratio and the last slope  $\geq 0$  then
8:   return not found
9: end if
10:  $t_{\text{left}} \leftarrow$  last observed time
11:  $t_{\text{right}} \leftarrow \text{max\_time} \times \text{horizon\_factor}$ 
12: for  $\text{iteration} \leftarrow 1$  to  $\text{search\_iter}$  do
13:   evaluate predict_ratio on a uniform grid from  $t_{\text{left}}$  to  $t_{\text{right}}$ 
14:   if predicted_ratio - target_ratio never changes sign then
15:     return not found
16:   end if
17:   shrink  $[t_{\text{left}}, t_{\text{right}}]$  to the first sign-change interval
18:   if the interval is narrower than time_atol then
19:     break
20:   end if
21: end for
22:  $t \leftarrow$  midpoint of  $[t_{\text{left}}, t_{\text{right}}]$ 
23: if  $|\text{predict\_ratio}(t) - \text{target\_ratio}| \leq \text{query\_tol\_factor} \times \text{tolerance}$  then
24:   return (found,  $t$ )
25: end if
26: return not found
```
